

Concurrency and Locks

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)

Thread — Flow of Instructions

CPU core가 하나면 여러 thread가 시분할로 나눠서 사용하지만
CPU core가 여러개면 실제 동시에 각각의 core에서 병렬 처리된다.

■ Concept

- A thread represents a *sequence of instructions* executed by the CPU.
- Define how a program's instructions flow through time — the path of execution.
- Each program may have one or more threads, each following its own control flow.

■ Structure

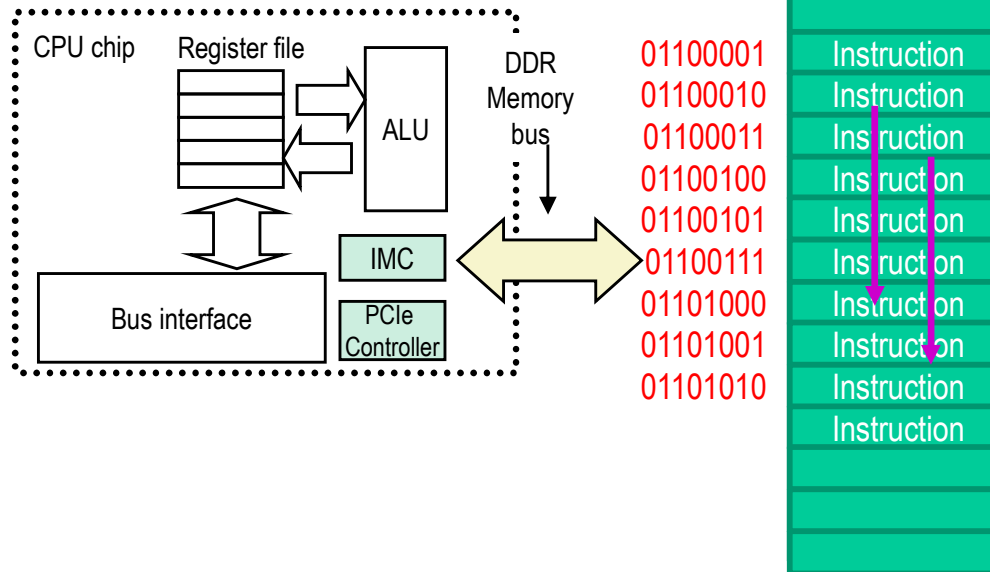
★ Heap은 공유됨

- Program: A static set of instructions (code).
- Thread: The *active execution flow* of those instructions.
- Each thread has:
 - Program Counter (PC): tracks the next instruction to execute
 - Stack: stores function calls and local variables
 - Registers: hold temporary computation data

각자
보유

Example

Two Threads
=
Two Execution
Flows



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "common.h"
4
5 volatile int counter = 0;
6 int loops;
7
8 void *worker(void *arg) {
9     int i;
10    for (i = 0; i < loops; i++) {
11        counter++;
12    }
13    return NULL;
14 }
15
16 int
17 main(int argc, char *argv[])
18 {
19     if (argc != 2) {
20         fprintf(stderr, "usage: threads
21             <value>\n");
22         exit(1);
23     }
24     loops = atoi(argv[1]);
25     pthread_t p1, p2;
26     printf("Initial value : %d\n", counter);
27     Pthread_create(&p1, NULL, worker, NULL);
28     Pthread_create(&p2, NULL, worker, NULL);
29     Pthread_join(p1, NULL);
30     Pthread_join(p2, NULL);
31     printf("Final value : %d\n", counter);
32     return 0;
33 }
```

<Program or Process>

- Threads within the same process **share** the same **address space**.
- Multiple threads can run different execution flows (or code paths) simultaneously within the same program.
- But, each thread has its own Program Counter (PC), leading to multiple PCs within a multi-threaded process.

Concurrency Problem

하나의 Process안에 여러 thread로 실행된다면 Heap을 공유하지만, 다른(부모/자식) Process에서 thread가 각각 실행된다면 data 공유가 불가능하기 때문에 IPC(Inter-Process Communication)을 사용해야 한다.

- The OS handles many processes simultaneously by rapidly switching between them.
- This time-sharing creates the illusion that programs run at the same time.
- In **multi-threaded programs**, multiple threads within one process run concurrently.
- Such parallel execution can cause concurrency issues like **race conditions**.

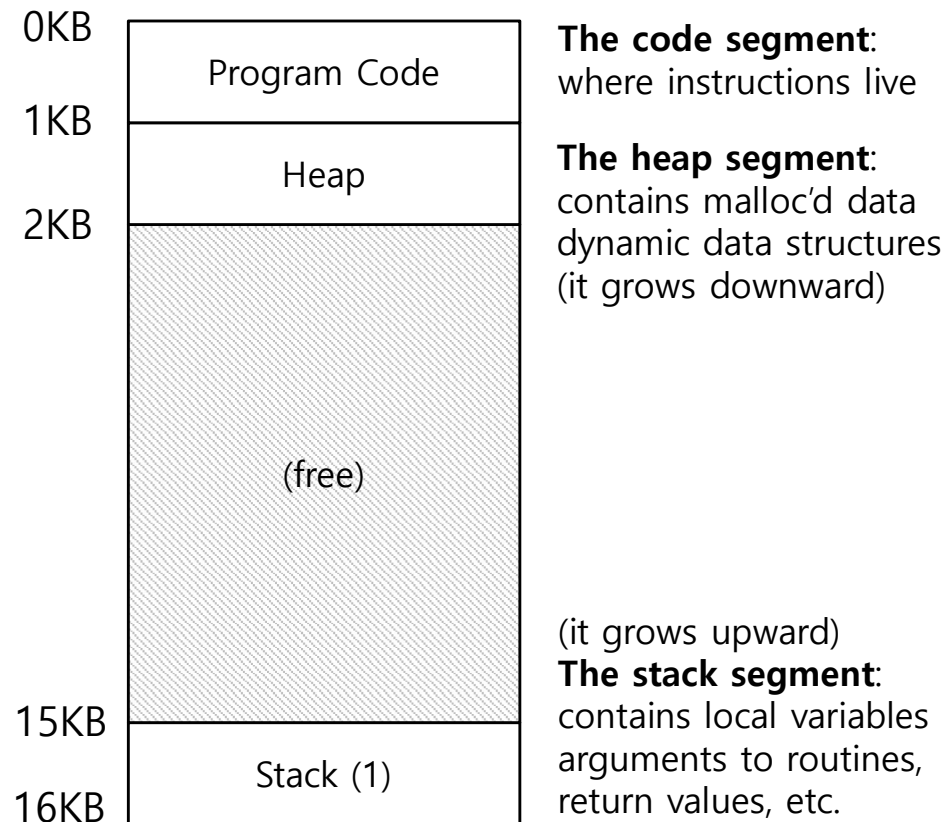
Context Switch between Threads

- Each thread maintains its own Program Counter (PC) and a private set of registers.
 - One or more Thread Control Blocks (TCBs) are required to store the state of each thread.
- When context switching from T1 to T2,
 - 1) the state of T1 is saved into its TCB, and
 - 2) the state of T2 is restored from its TCB.

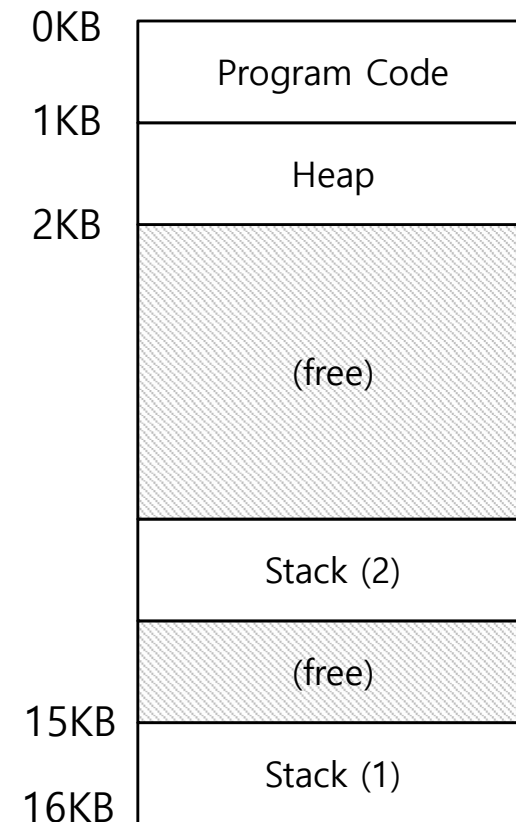
(Note) T1 and T2 uses the same **address space**.

Thread-Specific Stacks

- Each thread possesses its own dedicated stack space.
 - The stack stores the execution context (function calls, local variables) specific to that particular thread.



**A Single-Threaded
Address Space**



**Two threaded
Address Space**

Why Use Threads?

■ Parallelism

- Threads are essential for taking advantage of modern multi-core processors.

Feature	Single-Threaded Program	Multi-Threaded Program
Execution	The task flow is straightforward but sequential and slow.	The typical way to make programs run faster on modern hardware (simultaneous execution)

■ **Avoid I/O Blocking: Prevents program stalls caused by slow I/O.**

- **Overlap Activities:** Threading enables the parallel execution (overlap) of I/O operations and CPU tasks within a single program.
- **Analogy:** Similar to how multiprocessing managed overlapping processes across different programs.

An Example: Thread Creation

■ Simple Thread Creation Code (t0.c)

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include "common.h"
5  #include "common_threads.h"
6
7  void *mythread (void *arg) {
8      printf ("%s\n", (char *) arg);
9      return NULL;
10 }
11
12 int main (int argc, char *argv[]) {
13     pthread_t p1, p2;
14     int rc;
15     printf("main: begin\n");
16     Pthread_create(&p1, NULL, mythread, "A");
17     Pthread_create(&p2, NULL, mythread, "B");
18     // join waits for thre threads to finish
19     Pthread_join(p1, NULL);
20     Pthread_join(p2, NULL);
21     printf("main: end\n");
22     return 0;
23 }
```


Thread Trace (1)

main	Thread 1	Thread 2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
waits for T1		
	runs	
	prints "A"	
	returns	
waits for T2		
		runs
		prints "B"
		returns
prints "main: end"		

Thread Trace (2)

main	Thread 1	Thread 2
starts running		
prints "main: begin"		
creates Thread 1		
	runs	
	prints "A"	
	returns	
creates Thread 2		
		runs
		prints "B"
		returns
waits for T1		
returns immediately; T1 is done		
waits for T2		
returns immediately; T2 is done		
prints "main: end"		

Thread Trace (3)

main	Thread 1	Thread 2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
		runs
		prints "B"
		returns
waits for T1		
	runs	
	prints "A"	
	returns	
waits for T2		
returns immediately; T2 is done		
prints "main: end"		

Race condition

■ Increasing a value of a variable

- counter = counter + 1

여러 thread가 동시에 실행할 때 경합(race)하는 치명적인 이슈가 발생한다.

```
105    mov 0x8049a1c, %eax
108    add $0x1, %eax
113    mov %eax, 0x8049a1c
```

Race condition

■ Example with two threads

- counter = counter + 1 (default is 50)
- We expect the result is 52. However,

1.Thread1이 값을 가져와서 증가시킴
2.Thread2가 값을 가져와서 증가시킨 후 저장함
3.Thread1이 값을 저장함

OS	Thread1	Thread2	(after instruction)		
			PC	%eax	counter
	<i>before critical section</i>		100	0	50
	mov 0x8049a1c, %eax		105	50	50
	add \$0x1, %eax		108	51	50
interrupt	save T1's state				
	restore T2's state		100	0	50
		mov 0x8049a1c, %eax	105	50	50
		add \$0x1, %eax	108	51	50
		mov %eax, 0x8049a1c	113	51	51
interrupt	save T2's state				
	restore T1's state		108	51	50
	mov %eax, 0x8049a1c		113	51	51

위와 같은 현상이 왜 발생하는가? => Interrupt

Terminologies

Critical Section에서는 Lock을 사용해서 동시성을 제어해야 한다.

=> Lock은 한번에 하나의 thread만 얻을 수 있기 때문에 안전하게 저장 작업까지 실행할 수 있다.

■ Race Condition

- A situation where the outcome of concurrent threads depends on the specific order or timing of their execution.
- The results are non-deterministic and can vary with each run.
- Leads to an indeterminate or inconsistent final result in shared data.

■ Critical Section

- A segment of code within a concurrent program that accesses shared variables or resources.
- Requirement: It must not be executed by more than one thread or process simultaneously (concurrently).
 - Multiple threads executing the critical section at the same time will result in a race condition.
- Solution: Requires mechanisms like mutual exclusion to guarantee atomicity (indivisible execution) within the critical section.

The Wish for Atomicity

- **Ideal Approach:** Make the increment operation a single, indivisible assembly instruction.

```
memory-add 0x8049a1c, $0x1
```

- **Reality:** We generally lack such a dedicated atomic instruction for complex operations.
- **Solution:** Use synchronization mechanisms, such as locks, to achieve atomicity.
- **Goal:** Ensure that an entire critical section executes as a single, atomic unit (execute a series of instructions atomically).

```
1  lock_t mutex;  
2  . . .  
3  lock(&mutex);  
4  counter = counter + 1;  
5  unlock(&mutex);
```



Critical section

Thread API

Thread Creation

```
#include <pthread.h>

int
pthread_create(      pthread_t*      thread,
                    const pthread_attr_t* attr,
                    void*             (*start_routine) (void*),
                    void*             arg);
```

- **thread**: Used to interact with this thread.
- **attr**: Used to specify any attributes this thread might have.
 - Stack size, Scheduling priority, ...
- **start_routine**: the function this thread start running in.
- **arg**: the argument to be passed to the function (start routine)
 - *a void pointer* allows us to pass in *any type of* argument.

Creating a Thread

```
#include <pthread.h>

typedef struct __myarg_t {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc;

    myarg_t args;
    args.a = 10;
    args.b = 20;
    rc = pthread_create(&p, NULL, mythread, &args);
    ...
}
```

Wait for a thread to complete

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- thread: Specify which thread *to wait for*.
- value_ptr: A pointer to the return value
 - Because `pthread_join()` routine changes the value, you need to **pass the pointer** to that value.

Waiting for Thread Completion

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <assert.h>
4  #include <stdlib.h>
5
6  typedef struct __myarg_t {
7      int a;
8      int b;
9  } myarg_t;
10
11 typedef struct __myret_t {
12     int x;
13     int y;
14 } myret_t;
15
16 void *mythread(void *arg) {
17     myarg_t *m = (myarg_t *) arg;
18     printf("%d %d\n", m->a, m->b);
19     myret_t *r = malloc(sizeof(myret_t));
20     r->x = 1;
21     r->y = 2;
22     return (void *) r;
23 }
24
```

Waiting for Thread Completion (Cont.)

```
25  int main(int argc, char *argv[]) {
26      int rc;
27      pthread_t p;
28      myret_t *m;
29
30      myarg_t args;
31      args.a = 10;
32      args.b = 20;
33      pthread_create(&p, NULL, mythread, &args);
34      pthread_join(p, (void **) &m); // this thread has been
                                     // waiting inside of the
                                     // pthread_join() routine.
35      printf("returned %d %d\n", m->x, m->y);
36      return 0;
37  }
```

Example: Dangerous code

■ Exercise caution when returning values from a thread

```
1 void *mythread(void *arg) {
2     myarg_t *m = (myarg_t *) arg;
3     printf("%d %d\n", m->a, m->b);
4     myret_t *r = malloc(sizeof(myret_t));
5     r.x = 1;
6     r.y = 2;
7     return (void *) &r;
8 }
```

malloc()으로 heap을 생성해서 주소값을 r에 넣어둔 상태에서 r의 참조를 return 하면 heap의 주소가 리턴되는 것과 같다.

=> 그래서 heap에서 x, y값을 가져올 수 있음

```
1 void *mythread(void *arg) {
2     myarg_t *m = (myarg_t *) arg;
3     printf("%d %d\n", m->a, m->b);
4     myret_t r; // ALLOCATED ON STACK: BAD!
5     r.x = 1;
6     r.y = 2;
7     return (void *) &r;
8 }
```

Dangerous Code

heap을 생성하지 않고 그냥 r변수에 x, y를 담아서 r의 참조를 리턴하면.. 지역변수인 r은 mythread()함수가 종료되면 소멸된다.

=> 그렇기 때문에 쓰레기값이 나옴.

- Local variables (like 'r') reside on the stack and are automatically de-allocated when the function exits.

Simpler Argument Passing to a Thread

■ Just passing in a single value

```
1  void *mythread(void *arg) {
2      int m = (int) arg;
3      printf("%d\n", m);
4      return (void *) (arg + 1);
5  }
6
7  int main(int argc, char *argv[]) {
8      pthread_t p;
9      int rc, m;
10     pthread_create(&p, NULL, mythread, (void *) 100);
11     pthread_join(p, (void **) &m);
12     printf("returned %d\n", m);
13     return 0;
14 }
```

Locks and Mutual Exclusion

- Locks provide mutual exclusion to a **critical section**.

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- Example

```
pthread_mutex_t lock;  
pthread_mutex_lock(&lock);  
x = x + 1; // or whatever your critical section is  
pthread_mutex_unlock(&lock);
```

- If no other thread currently holds the lock, the current thread will **acquire** the lock and enter the critical section immediately.
- If another thread already holds the lock, the current thread calling the lock function will **wait** (or "not return from the call") until it successfully acquires the lock.

Locks and Mutual Exclusion (Cont')

- All locks must be **properly initialized**.

- One way: using `PTHREAD_MUTEX_INITIALIZER`

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

- The dynamic way: using `pthread_mutex_init()`

```
int rc = pthread_mutex_init(&lock, NULL);  
assert(rc == 0); // always check success!
```

Locks and Mutual Exclusion (Cont')

■ Check errors code when calling lock and unlock

- An example wrapper

```
// Use this to keep your code clean but check for failures
// Only use if exiting program is OK upon failure
void Pthread_mutex_lock(pthread_mutex_t *mutex) {
    int rc = pthread_mutex_lock(mutex);
    assert(rc == 0);
}
```

■ These two calls are used in lock acquisition

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_timelock(pthread_mutex_t *mutex,
                           struct timespec *abs_timeout);
```

- trylock: return failure if the lock is already held — 실패했을 때 다시 요청.
- timelock: return after a timeout — 특정 시간 이후 다시 요청.

Locks

Locks: The Basic Idea

- Ensure that any critical section executes as if it were **a single atomic instruction**.

- An example: the canonical update of a shared variable

```
balance = balance + 1;
```

- Add some code around the critical section

```
1  lock_t mutex; // some globally-allocated lock 'mutex'
2  ...
3  lock(&mutex);
4  balance = balance + 1;
5  unlock(&mutex);
```

The Lock Variable (State)

- **Purpose:** A lock variable holds the current state of a lock.

- **Possible States:**
 - **Available** (or unlocked, free):
 - No thread currently holds the lock.
 - The critical section is accessible.
 - **Acquired** (or locked, held):
 - Exactly one thread holds the lock.
 - That thread is presumably executing within the critical section.

Semantics of the lock()

- **When a thread calls `lock()`, it attempts to acquire the lock.**
 - Success Condition: If no other thread currently holds the lock, the calling thread acquires it immediately and enters the critical section.
 - Ownership: The thread that successfully acquires the lock is designated as the owner of the lock.
- **Mutual Exclusion:**
 - While the owner thread is inside the critical section, all other threads attempting to call `lock()` are prevented from entering.
 - They must wait until the lock is released.

[상호 배제]

유권이 있는 스레드가 임계 영역(Critical Section) 안에 있는 동안, `lock()`을 호출하여 진입하려는 다른 모든 스레드들은 들어오지 못하게 차단된다.

POSIX Threads (Pthreads) Locks: Mutex

- **Mutex is the name used by the POSIX library for a standard lock (for short, Mutual Exclusion)**

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3  Pthread_mutex_lock(&lock); // wrapper for pthread_mutex_lock()
4  balance = balance + 1;
5  Pthread_mutex_unlock(&lock);
```

- **Purpose:**

- Mutexes are used to provide mutual exclusion and safely synchronize access to shared resources between threads.

- **Advanced Usage:**

- We can use different locks to protect different shared variables.
- This approach, known as fine-grained locking, increases overall concurrency within the system.

Building a Lock

■ Goal:

- The objective is to create efficient locks that provide mutual exclusion at a low performance cost.

■ Requirement:

- Building robust and efficient locks requires essential support from both the underlying hardware and the operating system (OS).

Evaluating Locks: Basic Criteria

■ Mutual Exclusion (Correctness):

- Does the lock work correctly?
- Does it effectively prevent multiple threads from entering the critical section simultaneously?

■ Fairness (Liveness): 혼자만 Lock를 독식하거나 starvation이 발생하면 안된다.

- Does each thread contending for the lock get a fair opportunity to acquire it once it becomes free?
- Does the lock prevent starvation (a scenario where a thread waits indefinitely to acquire the lock)?

■ Performance: Lock을 잡고 관리하는데에도 리소스가 들어간다.

- What are the time overheads added by using the lock mechanism?
- This includes performance under both low contention (single thread) and high contention (multiple threads waiting) scenarios.

Controlling Interrupts for Mutual Exclusion

■ Disabling interrupts for the duration of a critical section.

- **Historical Context:** One of the earliest solutions used to provide mutual exclusion, primarily invented for single-processor systems.

```
1  void lock() {  
2      DisableInterrupts();  
3  }  
4  void unlock() {  
5      EnableInterrupts();  
6  }
```

■ Problems & Limitations:

- **Trust Issues:** Requires too much trust in application programs; a greedy or malicious program could monopolize the processor by never re-enabling interrupts.
- **Multiprocessor Failure:** Does not work on modern multiprocessor systems. Disabling interrupts on one CPU has no effect on another CPU, which can still access the critical section.
- **Performance Overhead:** Code that masks or unmask interrupts tends to be executed slowly by modern CPUs due to complex instruction pipelines that must be flushed.

Why Hardware Support is Needed

- **First Attempt (Software Flag):** We can attempt to use a simple flag variable to denote whether the lock is held or not.
- **Problem:**
 - Purely software-based lock implementations are generally insufficient and contain flaws, especially in modern multiprocessor systems.

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 → lock is available, 1 → held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1; // now SET it !
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

Why Hardware Support is Needed (Cont')

- **Problem 1:** No Mutual Exclusion (assume `flag=0` to begin)

Thread1	Thread2
<pre>call lock() while (flag == 1) interrupt: switch to Thread 2</pre>	<pre>call lock() while (flag == 1) flag = 1; interrupt: switch to Thread 1</pre>
<pre>flag = 1; // set flag to 1 (too!)</pre>	

- **Problem 2:** Spin-waiting wastes time waiting for another thread.
- **So, we need an atomic instruction supported by Hardware!**
 - *test-and-set* instruction, also known as *atomic exchange*

Hardware Support: Test and Set (Atomic Exchange)

- **TestAndSet** is a specialized hardware instruction designed to support the creation of simple, atomic locks.
 - Perform two actions simultaneously as a single, indivisible operation:
 - **Test/Return:** It returns the old value stored at a specific memory address (pointer).
 - **Set/Update:** It simultaneously updates that same memory location to a new value.

```
1  int TestAndSet(int *ptr, int new) {  
2      int old = *ptr;    // fetch old value at ptr  
3      *ptr = new;        // store 'new' into ptr  
4      return old;        // return the old value  
5  }
```

- This entire sequence of operations is guaranteed to be performed atomically by the hardware, preventing any other CPU or thread from interrupting the process.

A Simple Spin Lock using TestAndSet

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0 indicates that lock is available,
7      // 1 that it is held
8      lock->flag = 0;
9  }
10
11 void lock(lock_t *lock) {
12     while (TestAndSet(&lock->flag, 1) == 1)
13         ;           // spin-wait
14 }
15
16 void unlock(lock_t *lock) {
17     lock->flag = 0;
18 }
```

- **Note:** To work correctly on *a single processor*, it requires a preemptive scheduler.

Evaluating Spin Locks

■ Correctness (Mutual Exclusion): Yes

- The lock correctly ensures that only a single thread can enter the critical section at any given time, thanks to atomic hardware instructions (like `TestAndSet`).

■ Fairness: No

- Spin locks do not provide any fairness guarantees.
- A thread that is "spinning" (repeatedly checking the lock state in a loop) may potentially spin forever (known as starvation) if other threads continually acquire and release the lock before the waiting thread gets a chance.

■ Performance:

- Single CPU Systems: Performance overheads can be quite painful because the spinning thread wastes valuable CPU cycles that could be used by other processes.
- Multi-CPU Systems: Spin locks work reasonably well if the number of threads waiting for the lock roughly equals the number of available CPUs (i.e., each thread gets its own CPU to spin on).

Concurrency and Locks: Summary

■ Problem:

- When multiple threads execute concurrently, a race condition can occur, leading to unpredictable and incorrect results. To prevent this, we must protect critical sections—the parts of the code that access shared resources.

■ Solution:

- Use locks to enforce mutual exclusion, ensuring that only one thread can be inside a critical section at any given time.

■ Implementation:

- Building a robust lock requires essential support from both the hardware (via atomic instructions like `TestAndSet`) and the operating system (for more efficient locks that do not waste CPU cycles).

■ Evaluation:

- Correctness: Does it successfully provide mutual exclusion?
- Fairness: Does every waiting thread get a fair chance to acquire the lock (preventing starvation)?
- Performance: What is the overhead added by using the lock under different contention levels?